



Εθνικό Μετσόβιο Πολυτεχνείο

Σχολή Ηλεκτρολόγων Μηχανικών και Μηχανικών Υπολογιστών

Τομέας Τεχνολογίας Πληροφορικής και Υπολογιστών

Εργαστήριο Βάσεων Γνώσεων και Δεδομένων

Προχωρημένα Θέματα Βάσεων Δεδομένων

Εξαμηνιαία Εργασία

Πέτρος Βιτάλης Α.Μ.: 03121117

Κιμίνου Φραγκίσκα Α.Μ.: 03121009

Github link: <https://github.com/fragkiskakim/Advanced-Databases-Project>

Ακαδημαϊκό Έτος: 2025-2026

Διδάσκων: Δημήτριος Τσουμάκος

Υπεύθυνος Εργαστηρίου: Νικόλαος Χαλβαντζής

Δεκέμβριος, 2025

Query 1

Να ταξινομηθούν, σε φθίνουσα σειρά, οι ηλικιακές ομάδες των θυμάτων σε περιστατικά που περιλαμβάνουν οποιαδήποτε μορφή “βαριάς σωματικής βλάβης”. Θεωρείστε τις εξής ηλικιακές ομάδες:

- Παιδιά: < 18
- Ενήλικοι: 25– 64
- Νεαροί ενήλικοι: 18– 24
- Ηλικιωμένοι: >64

Για την εκτέλεση χρησιμοποιήθηκαν τρεις διαφορετικές προσεγγίσεις του Apache Spark:

1. **DataFrame API χωρίς UDF** (χρήση built-in Spark SQL συναρτήσεων)
2. **DataFrame API με UDF** (χρήση Python User-Defined Function)
3. **RDD API** (χαμηλού επιπέδου λειτουργίες map/filter/reduce)

Όλες οι υλοποιήσεις εκτελέστηκαν με:

- 4 executors
- 1 core ανά executor
- 2 GB μνήμη ανά executor

Αποτελέσματα Query 1

Και οι τρεις υλοποιήσεις παρήγαγαν τα παρακάτω αποτελέσματα:

Age_Group	count
Ενήλικοι	121660
Νεαροί ενήλικες	33758
Παιδιά	16014
Ηλικιωμένοι	6011

Χρόνοι Εκτέλεσης

Οι χρόνοι εκτέλεσης που μετρήθηκαν για την κάθε υλοποίηση παρουσιάζονται παρακάτω:

Υλοποίηση	Χρόνος (sec)
DataFrame χωρίς UDF	4.20
DataFrame με UDF	4.97
RDD API	27.47

Παρατηρήσεις

1. Για αυτό το query χρησιμοποιήσαμε μόνο το LA_Crime_Data.csv το οποίο περιέχει τις πληροφορίες που θέλουμε για τις ηλικίες των θυμάτων και τον τύπο του εγκλήματος “aggravated assault”.
2. Η μέθοδος **DataFrame χωρίς UDF** είναι η γρηγορότερη υλοποίηση, καθώς χρησιμοποιεί Spark SQL functions και επιτρέπει στο Catalyst optimizer να κάνει επιθετική βελτιστοποίηση. Επίσης, εκμεταλλεύεται columnar execution, χρησιμοποιεί τον Tungsten engine για μνήμη και CPU optimization, και αποφεύγει serialization bottlenecks.
3. Η μέθοδος **DataFrame με UDF** είναι ελαφρώς πιο αργή, γιατί οι Python user defined functions δεν επιτρέπουν στη βελτιστοποίηση του Catalyst να λειτουργήσει και οδηγούν σε σπάσιμο του vectorized execution.
4. Τέλος, η μέθοδος **RDD API** είναι πολύ αργή με σημαντικά μεγαλύτερο χρόνο. Αυτό είναι αναμενόμενο, καθώς τα RDDs δεν ενδείκνυνται για analytics tasks, στα οποία τα DataFrames υπερτερούν. Τα RDDs χρησιμοποιούνται κυρίως όταν απαιτείται πολύ χαμηλού επιπέδου χειρισμός.

Query 2

Ανά έτος, να βρεθούν τα 3 φυλετικά γκρουπ με τα περισσότερα θύματα καταγεγραμμένων εγκλημάτων (Vict Descent) στο Los Angeles. Τα αποτελέσματα να εμφανιστούν με φθίνουσα σειρά αριθμού θυμάτων ανά φυλετικό γκρουπ – να υπολογιστεί και να εμφανιστεί επίσης το ποσοστό επί του συνολικού αριθμού θυμάτων ανά περίπτωση.

Για την εκτέλεση χρησιμοποιήθηκαν δύο διαφορετικές προσεγγίσεις του Apache Spark:

1. **DataFrame API**
2. **SQL API με UDF**

Και οι 2 υλοποιήσεις εκτελέστηκαν με:

- 4 executors
- 1 core ανά executor
- 2 GB μνήμη ανά executor

Αποτελέσματα Query 2

Και οι δύο υλοποιήσεις παρήγαγαν τα παρακάτω αποτελέσματα. Ενδεικτικά, παραθέτουμε για τα έτη 2025- 2022:

Year	Victim Descent	Count	Total per Year	Percentage (%)	Rank
2025	Hispanic/Latin/Mexican	34	84	40.5	1
2025	Unknown	24	84	28.6	2
2025	White	13	84	15.5	3
2024	Hispanic/Latin/Mexican	28.576	98.363	29.1	1
2024	White	22.958	98.363	23.3	2
2024	Unknown	19.984	98.363	20.3	3
2023	Hispanic/Latin/Mexican	69.401	200.846	34.6	1
2023	White	44.615	200.846	22.2	2
2023	Black	30.504	200.846	15.2	3
2022	Hispanic/Latin/Mexican	73.111	205.164	35.6	1
2022	White	46.695	205.164	22.8	2
2022	Black	34.634	205.164	16.9	3

Χρόνοι Εκτέλεσης

Οι χρόνοι εκτέλεσης που μετρήθηκαν για την κάθε υλοποίηση παρουσιάζονται παρακάτω:

Υλοποίηση	Χρόνος (sec)
DataFrame	5.11
SQL	6.53

Παρατηρήσεις

1. Για αυτό το query χρησιμοποιήσαμε το LA_Crime_Data.csv συνδυαστικά με το RE_codes.csv, το οποίο περιέχει τις πληροφορίες για τους κωδικούς των φυλετικών προφίλ που αντιστοιχούν στα αρχικά γράμματα του crime_data dataset. Κάναμε join στα δύο csvs για να έχουμε την συνολική πληροφορία.
2. Η μέθοδος **DataFrame** είναι ξανά η γρηγορότερη υλοποίηση, καθώς μεταφράζεται κατευθείαν σε Spark Logical plan και έχει άμεση χρήση των Catalyst optimizations.
3. Αντίθετα, η μέθοδος **SQL API** είναι ελαφρώς πιο αργή, γιατί το sql string που γράφουμε πρέπει να γίνει parse προκειμένου να φτιαχτεί το AST δέντρο και να γίνει η ανάλυση και τα δυνατά optimizations.
4. Ωστόσο, και οι δύο υλοποιήσεις αναμένεται να παράξουν πανομοιότυπο execution plan, αλλά η διαφορά στους χρόνους οφείλεται αποκλειστικά στο αρχικό στάδιο προετοιμασίας.

Query 3

Να ταξινομηθούν και να εμφανιστούν με φθίνουσα σειρά συχνότητας εμφάνισης οι μέθοδοι διάπραξης εγκλημάτων και οι αντίστοιχοι κωδικοί τους (Mocodes). Χρησιμοποιήστε το σύνολο MO Codes για να αντιστοιχίσετε τους κωδικούς με τις περιγραφές τους.

Για την εκτέλεση χρησιμοποιήθηκαν δύο διαφορετικές προσεγγίσεις του Apache Spark:

1. **DataFrame API**

2. **RDD API**

Και οι 2 υλοποιήσεις εκτελέστηκαν με:

- 4 executors
- 1 core ανά executor
- 2 GB μνήμη ανά executor

Αποτελέσματα Query 3

Και οι δύο υλοποιήσεις παρήγαγαν τα παρακάτω αποτελέσματα. Ενδεικτικά, παραθέτουμε τις πρώτες 5 συχνότερες μεθόδους διάπραξης εγκλημάτων:

MO Code	Description	Count
0344	Removes vict property	1,002,900
1822	Stranger	548,422
0416	Hit-Hit w/ weapon	404,773
0329	Vandalized	377,536
0913	Victim knew Suspect	278,618

Χρόνοι Εκτέλεσης

Οι χρόνοι εκτέλεσης που μετρήθηκαν για την κάθε υλοποίηση παρουσιάζονται παρακάτω:

Υλοποίηση	Χρόνος (sec)
DataFrame με Broadcast Hash Join (default method)	9.417
DataFrame με Broadcast Join	7.87
DataFrame με Shuffle Hash	9.361
DataFrame με Merge	7.594
DataFrame με Shuffle Replicate NL	7.637
RDD API	27.09

Παρατηρήσεις

1. Για την υλοποίηση του Query 3 αξιοποιήθηκε ο συνδυασμός του LA_Crime_Data.csv με το MO_codes.txt, το οποίο περιέχει περιγραφικές πληροφορίες για κάθε MO code. Αφού μετασχηματίστηκε το MO_codes.txt σε structured DataFrame, το πεδίο *Mocodes* του crime dataset έγινε explode ώστε κάθε MO code να αντιστοιχεί σε ξεχωριστή γραμμή, και στη συνέχεια πραγματοποιήθηκε join των δύο DataFrames. Με αυτό το σχήμα εκτελέστηκαν και μετρήθηκαν διαφορετικές στρατηγικές join του Spark SQL, καθώς και η αντίστοιχη υλοποίηση με RDDs.
2. Αρχικά, με χρήση της μεθόδου explain παρατηρούμε ότι το Spark Catalyst αποφάσισε αυτόματα να χρησιμοποιήσει **Broadcast Hash Join**. Ο λόγος είναι ότι το mo_df είναι μικρό και χωρά στη μνήμη κάθε executor. Το Broadcast Hash Join:
 - a. μεταδίδει το μικρό table σε όλους τους executors
 - b. αποφεύγει shuffle
 - c. έχει χαμηλό latency

Η επίδοση αυτής της μεθόδου είναι καλή αλλά όχι η βέλτιστη, γιατί εξαρτάται από το πόσο μεγάλο είναι το άλλο input (mo_counts) και τον φόρτο των executors.

3. Η επιβολή της μεθόδου **Broadcast Join** οδήγησε σε ακόμη καλύτερο χρόνο. Αυτό συνέβη επειδή:
 - a. το Spark εξαναγκάστηκε να μεταδώσει το μικρότερο DataFrame,
 - b. αποφεύχθηκε οποιοδήποτε shuffle για το δεύτερο table,
 - c. όλη η πράξη εκτελέστηκε με local hash matching.

Γνωρίζουμε ότι το broadcast join είναι εξαιρετικά αποτελεσματικό όταν το ένα table είναι μικρό και οι executors έχουν αρκετή μνήμη.

4. Ο **Shuffle Hash Join** έχει παρόμοιο χρόνο με το default. Απαιτεί:
 - a. shuffle των δύο DataFrames
 - b. repartition με hash
 - c. κατασκευή hash tables σε κάθε partition

Είναι αποτελεσματικός όταν τα tables είναι σχετικά μικρά ή όταν η join key έχει καλή κατανομή. Ωστόσο, το shuffle είναι δαπανηρό και το overhead είναι μεγαλύτερο από το broadcast. Γι' αυτό η επίδοση δεν ξεπέρασε την broadcast εκδοχή.

5. Το **Sort – Merge Join** είχε από τις καλύτερες επιδόσεις. Ο αλγόριθμος κάνει sort και των δύο πινάκων ως προς το join key και εκτελεί merge στο sorted input. Αποδίδει πολύ καλά, όταν τα δεδομένα είναι ήδη partitioned ή sorted, το join γίνεται σε μεγάλα datasets, ή υπάρχουν αρκετοί executors για παράλληλη ταξινόμηση. Στο δικό μας ερώτημα είναι η ταχύτερη μέθοδος, επειδή τα mo_counts και mo_df είναι σχετικά μικρά και πλήρως parallelizable στους 4 executors που χρησιμοποιήθηκαν.
6. Το **Shuffle Replicate NL Join** είχε επίσης πολύ καλή επίδοση. Σύμφωνα με τον μηχανισμό του, το μικρότερο table αντιγράφεται σε όλα τα partitions και το join εφαρμόζεται σαν nested loops. Είναι εξαιρετικός όταν το ένα table είναι πολύ μικρό ή το δεύτερο έχει μεγάλες skewed κατανομές.

Η επίδοσή του είναι συγκρίσιμη με τη sort-merge, αλλά ελαφρώς χειρότερη επειδή οι nested loop συγκρίσεις είναι ακριβότερες.

7. Η **RDD υλοποίηση** ήταν μακράν η πιο αργή (27.09 sec), περίπου 3–4 φορές πιο αργή από τις καλύτερες DataFrame solutions.

Αυτό είναι αναμενόμενο, γιατί:

- δεν υπάρχει Catalyst optimizer
- δεν υπάρχουν columnar execution & Tungsten optimizations
- το join γίνεται σε RDDs μετά τη μετατροπή σε DataFrame, άρα υπάρχει overhead
- η διαχείριση shuffle είναι χειροκίνητη και όχι βελτιστοποιημένη

Τα RDDs ενδείκνυνται μόνο για πολύ χαμηλού επιπέδου εργασίες, όχι για queries με joins.

Συνολικά, οι δύο καλύτερες επιλογές για το συγκεκριμένο query είναι:

- Sort-Merge Join (ταχύτερο)
- Broadcast Hash Join / Broadcast Join (πολύ αποδοτικό επειδή υπάρχει μικρό table)

Query 4

Να υπολογιστεί, ανά αστυνομικό τμήμα, ο αριθμός εγκλημάτων που έλαβαν χώρα πλησιέστερα σε αυτό από οποιοδήποτε άλλο, καθώς και η μέση απόστασή του από τις τοποθεσίες όπου σημειώθηκαν τα συγκεκριμένα περιστατικά. Τα αποτελέσματα να εμφανιστούν ταξινομημένα κατά αριθμό περιστατικών, με φθίνουσα σειρά.

Για την υλοποίηση χρησιμοποιήθηκε το **DataFrame API** σε συνδυασμό με τη βιβλιοθήκη **Apache Sedona** για τους γεωχωρικούς υπολογισμούς.

Η υλοποίηση εκτελέστηκε τρεις φορές εφαρμόζοντας κλιμάκωση (scaling) στους διαθέσιμους πόρους (με σταθερά 2 executors), σύμφωνα με τα παρακάτω configurations:

- Config A: 1 core, 2 GB μνήμη (ανά executor)
- Config B: 2 cores, 4 GB μνήμη (ανά executor)
- Config C: 4 cores, 8 GB μνήμη (ανά executor)

Αποτελέσματα Query 4

Και οι τρεις εκτελέσεις παρήγαγαν τα ίδια δεδομένα, με το τμήμα του Hollywood να συγκεντρώνει τον μεγαλύτερο αριθμό περιστατικών. Ενδεικτικά, παραθέτουμε τις πρώτες 5 περιοχές με τα περισσότερα εγκλήματα:

DIVISION	Average Distance (km)	Crime Count
HOLLYWOOD	2.074	224,124

VAN NUYS	2.939	208,129
SOUTHWEST	2.191	189,119
WILSHIRE	2.593	186,383
77TH STREET	1.717	170,620

Χρόνοι Εκτέλεσης

Οι χρόνοι εκτέλεσης που μετρήθηκαν για την κάθε υλοποίηση παρουσιάζονται παρακάτω:

Configuration	Cores/Exec	RAM/Exec	Χρόνος (sec)
Config A	1	2 GB	40.146
Config B	2	4 GB	31.660
Config C	4	8 GB	30.280

Παρατηρήσεις

1. Για την υλοποίηση του Query 4 αξιοποιήθηκε ο συνδυασμός του LA_Crime_Data (εγκλήματα) με το LA_Police_Stations (αστυνομικά τμήματα). Αρχικά, πραγματοποιήθηκε καθαρισμός των δεδομένων με φιλτράρισμα των εγγραφών που είχαν συντεταγμένες (0,0) (Null Island). Στη συνέχεια, με τη χρήση της βιβλιοθήκης Apache Sedona, δημιουργήθηκαν γεωμετρικά σημεία (Points) για κάθε έγκλημα και κάθε αστυνομικό τμήμα, ώστε να υπολογιστεί η απόσταση (ST_DistanceSphere) κάθε εγκλήματος από όλα τα τμήματα και να επιλεγεί το πλησιέστερο μέσω Window Functions.
2. Με χρήση της μεθόδου explain, παρατηρούμε ότι ο Spark Catalyst Optimizer επέλεξε τη στρατηγική Broadcast Nested Loop Join. Αυτή η επιλογή κρίνεται βέλτιστη για τη συγκεκριμένη περίπτωση, καθώς ο πίνακας των αστυνομικών τμημάτων είναι εξαιρετικά μικρός (μόλις 21 εγγραφές). Το **Broadcast Nested Loop Join**:
 - a. **Μεταδίδει (Broadcast)** τον μικρό πίνακα των τμημάτων σε όλους τους executors.
 - b. Επιτρέπει τον υπολογισμό του **Cross Join** (καρτεσιανό γινόμενο) τοπικά σε κάθε partition, χωρίς να απαιτείται δαπανηρό shuffle του μεγάλου πίνακα των εγκλημάτων.
 - c. Ελαχιστοποιεί την κίνηση δεδομένων στο δίκτυο, εκτελώντας τους γεωχωρικούς υπολογισμούς στη μνήμη κάθε κόμβου.
3. Από την ανάλυση της κλιμάκωσης (scaling) με διαφορετικά configurations (1, 2 και 4 cores ανά executor), προκύπτει ότι το ερώτημα είναι έντονα CPU-intensive λόγω των υπολογισμών αποστάσεων. Συγκεκριμένα:

- a. Η μετάβαση από το 1 στα 2 cores επέφερε σημαντική μείωση του χρόνου εκτέλεσης (περίπου 21%), καθώς ο αυξημένος παραλληλισμός επιτάχυνε την επεξεργασία των partitions.
- b. Η περαιτέρω αύξηση από τα 2 στα 4 cores παρουσίασε φαινόμενα **φθίνουσας απόδοσης (diminishing returns)**, με οριακή μόνο βελτίωση (~4%).
- c. Αυτό υποδεικνύει ότι πέρα από ένα σημείο, η προσθήκη πόρων δεν μεταφράζεται γραμμικά σε ταχύτητα, πιθανώς λόγω overhead στη διαχείριση των νημάτων ή υπάρξης σειριακών τμημάτων στην εκτέλεση που δεν μπορούν να παραλληλοποιηθούν περαιτέρω.

Query 5

Χρησιμοποιώντας ως αναφορά τα δεδομένα της απογραφής του 2020 για τον πληθυσμό και τα οικονομικά στοιχεία του 2021 για το εισόδημα ανα νοικοκυριό, να υπολογίσετε μέσα στη διετία 2020-2021 τη συσχέτιση μέσου ετήσιου κατακεφαλήν εισοδήματος με την ετήσια μέση αναλογία εγκλημάτων ανά άτομο για κάθε περιοχή του Λος Άντζελες. Επαναλάβετε τον υπολογισμό εξετάζοντας μόνο τις 10 περιοχές με το υψηλότερο και τις 10 με το χαμηλότερο ετήσιο κατακεφαλήν εισόδημα.

Για την υλοποίηση του ερωτήματος συνδυάστηκαν το **DataFrame API**, που αξιοποιήθηκε για τη διαχείριση και τον γεωχωρικό συσχετισμό των δεδομένων (Spatial Join) με χρήση της βιβλιοθήκης **Apache Sedona**, και το **SQL API**, μέσω του οποίου εκτελέστηκαν οι απαραίτητες συναθροίσεις και υπολογισμοί.

Η υλοποίηση εκτελέστηκε χρησιμοποιώντας σταθερούς συνολικούς πόρους (8 cores και 16GB μνήμης στο cluster), κατανεμημένους με τρεις διαφορετικούς τρόπους στους executors, σύμφωνα με τα παρακάτω configurations:

1. Config A: 2 executors (4 cores, 8 GB μνήμη ανά executor)
2. Config B: 4 executors (2 cores, 4 GB μνήμη ανά executor)
3. Config C: 8 executors (1 core, 2 GB μνήμη ανά executor)

Αποτελέσματα Query 5

Η ανάλυση συσχέτισης (correlation) μεταξύ του μέσου ετήσιου εισοδήματος και της εγκληματικότητας παρήγαγε τα εξής αποτελέσματα:

Κατηγορία Περιοχών	Συσχέτιση
Όλες οι περιοχές	-0.283
Top 10 (Πλουσιότερες)	0.022
Bottom 10 (Φτωχότερες)	-0.709

Χρόνοι Εκτέλεσης

Οι μέσοι χρόνοι που προέκυψαν μετά από 5 εκτελέσεις για την κάθε υλοποίηση φαίνονται παρακάτω:

Configuration	Executors	Cores/Exec	RAM/Exec	Μέσος χρόνος (sec)
Config A	2	4	8 GB	77.11
Config B	4	2	4 GB	85.27
Config C	8	1	2 GB	79.05

Παρατηρήσεις

- Για την υλοποίηση του ερωτήματος πραγματοποιήθηκαν δύο ειδών joins:
 - Αρχικά, απαιτήθηκε Spatial Join (ST_Within) μεταξύ των σημείων τέλεσης εγκλημάτων και των πολυγώνων των περιοχών (Census Blocks). Όπως φαίνεται στο Physical Plan, το Spark χρησιμοποίησε τη στρατηγική **RangeJoin** (optimized spatial join από τη βιβλιοθήκη Sedona) για να συνδέσει αποδοτικά τα γεωμετρικά δεδομένα.
 - Επιπλέον, για τη σύνδεση των δημογραφικών στοιχείων με τα οικονομικά δεδομένα (join key: Zip Code), ο Catalyst Optimizer επέλεξε τη στρατηγική **Broadcast Hash Join**. Η επιλογή αυτή σχολιάζεται ως βέλτιστη, καθώς το dataset των εισοδημάτων είναι μικρό σε όγκο, επιτρέποντας την αποδοτική μετάδοσή του (broadcast) σε όλους τους executors χωρίς να απαιτείται shuffle.
- Σχετικά με την κλιμάκωση (scaling), κρατήσαμε σταθερούς τους συνολικούς πόρους (8 Cores, 16GB RAM) και μεταβάλαμε την κατανομή τους. Από τους χρόνους εκτέλεσης προκύπτουν τα εξής:
 - Config A (2 Executors, 4 Cores):** Πέτυχε τον βέλτιστο χρόνο (~77 sec). Η ύπαρξη περισσότερων πυρήνων ανά executor (vertical scaling) ελαχιστοποιεί την ανάγκη για μεταφορά δεδομένων μέσω δικτύου (network shuffle overhead) και ευνοεί την αποδοτικότερη χρήση των Broadcast variables.
 - Config B (4 Executors, 2 Cores):** Εμφάνισε τη χειρότερη επίδοση (~85 sec). Ο κατακερματισμός της μνήμης και το αυξημένο overhead διαχείρισης περισσότερων JVM processes δεν απέδωσε σε σχέση με το Config A.
 - Config C (8 Executors, 1 Core):** Πλησίασε την επίδοση του Config A (~79 sec). Αν και κάθε executor είχε ελάχιστους πόρους, το πλήθος των executors επέτρεψε υψηλό παραλληλισμό. Ωστόσο, δεν ξεπέρασε το Config A, επιβεβαιώνοντας ότι για βαριές υπολογιστικές διαδικασίες (όπως τα spatial joins), οι "παχύτεροι" executors (fat executors) συχνά αποδίδουν καλύτερα από πολλούς μικρούς (thin executors).